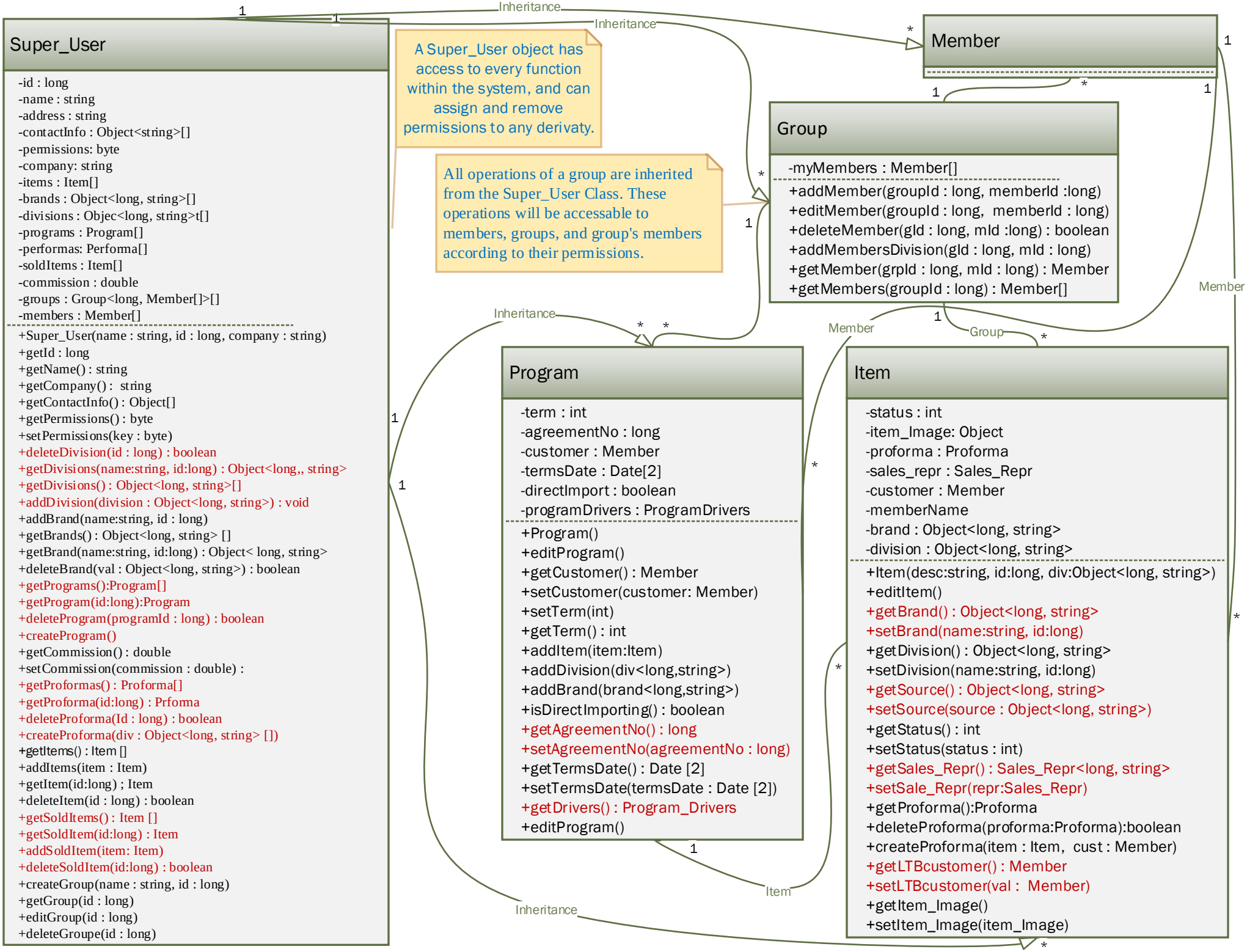
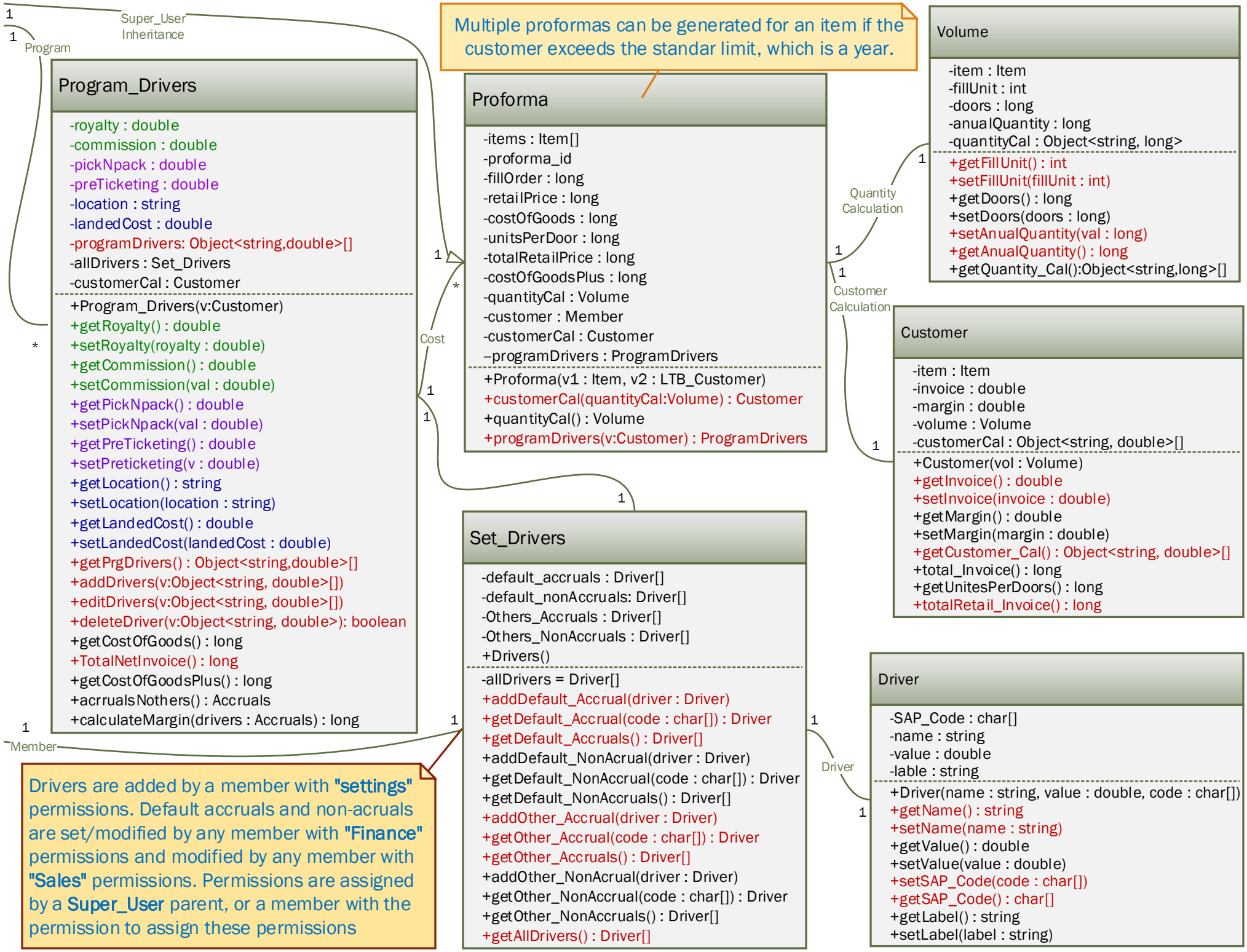
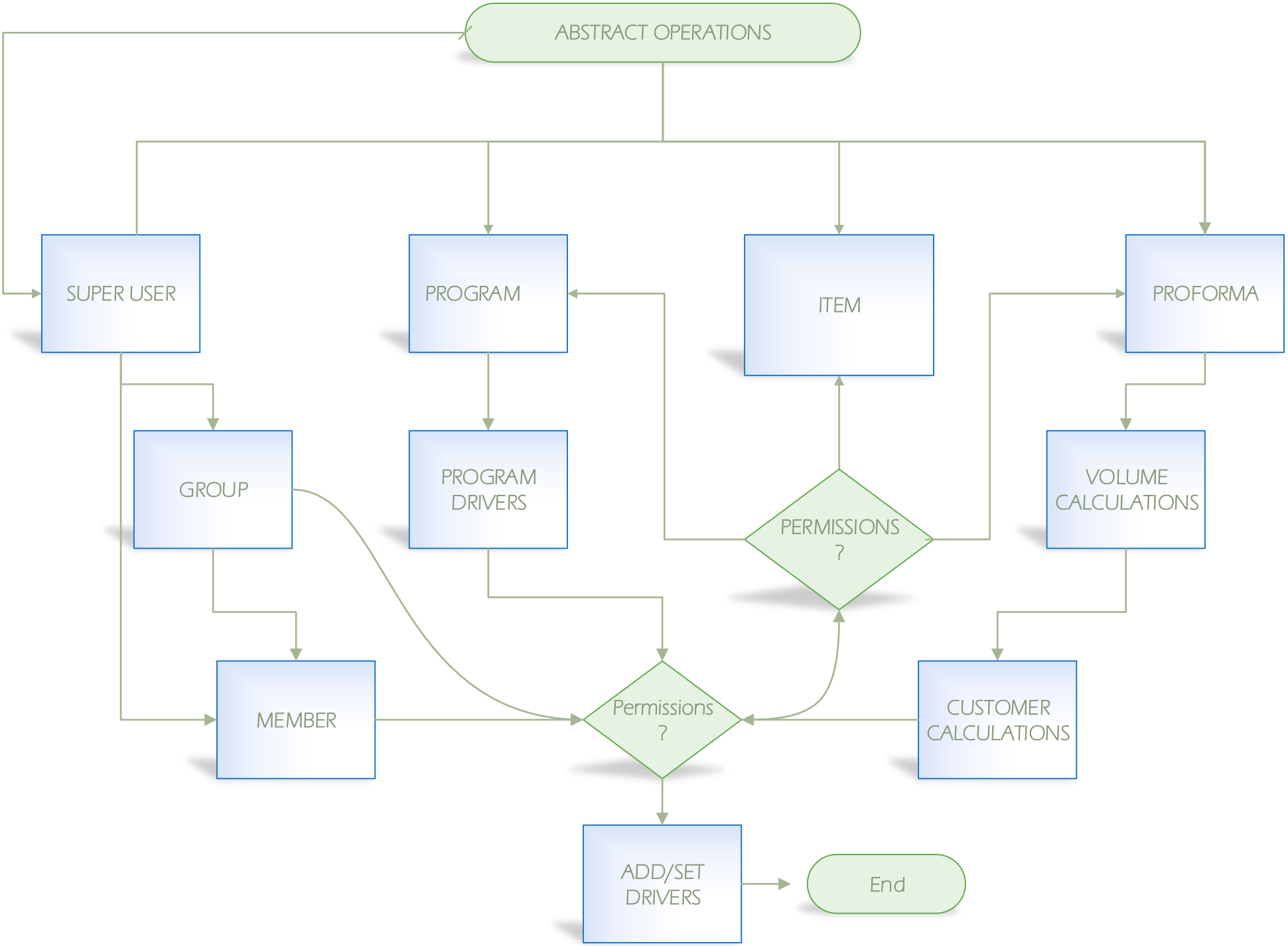

Super_User
-id : long
-name : string
-address : string
-contactInfo : Object<string>[]
-permissions: byte
-company: string
-items : Item[]
-brands : Object<long, string>[]
-divisions : Objec<long, string>t[]
-programs : Program[]
-performas: Performa[]
-soldItems : Item[]
-commission : double
-groups : Group<long, Member[]>[]
-members : Member[]
+Super_User(name : string, id : long, company : string)
+getId : long
+getName() : string
+getCompany() : string
+getContactInfo() : Object[]
+getPermissions() : byte
+setPermissions(key : byte)
+deleteDivision(id : long) : boolean
+getDivisions(name:string, id:long) : Object<long,, string>
+getDivisions() : Object<long, string>[]
+addDivision(division : Object<long, string>) : void
+addBrand(name:string, id : long)
+getBrands() : Object<long, string> []
+getBrand(name:string, id:long) : Object< long, string>
+deleteBrand(val : Object<long, string>) : boolean
+getPrograms():Program[]
+getProgram(id:long):Program
+deleteProgram(programId : long) : boolean
+createProgram()
+getCommission() : double
+setCommission(commission : double) :
+getProformas() : Proforma[]
+getProforma(id:long) : Prforma
+deleteProforma(Id : long) : boolean
+createProforma(div : Object<long, string> [])
+getItems() : Item []
+addItems(item : Item)
+getItem(id:long) ; Item
+deleteItem(id : long) : boolean
+getSoldItems() : Item []
+getSoldItem(id:long) : Item
+addSoldItem(item: Item)
+deleteSoldItem(id:long) : boolean
+createGroup(name : string, id : long)
+getGroup(id : long)
+editGroup(id : long)
+deleteGroupe(id : long)
A Super_User object has access to every function within the system, and can assign and remove permissions to any derivaty.
All operations of a group are inherited from the Super_User Class. These operations will be accessable to members, groups, and group's members according to their permissions.
Member
Group
-myMembers : Member[]
+addMember(groupId : long, memberId :long)
+editMember(groupId : long, memberId : long)
+deleteMember(gId : long, mId :long) : boolean
+addMembersDivision(gId : long, mId : long)
+getMember(grpId : long, mId : long) : Member
+getMembers(groupId : long) : Member[]
Program
-term : int
-agreementNo : long
-customer : Member
-termsDate : Date[2]
-directImport : boolean
-programDrivers : ProgramDrivers
+Program()
+editProgram()
+getCustomer() : Member
+setCustomer(customer: Member)
+setTerm(int)
+getTerm() : int
+addItem(item:Item)
+addDivision(div<long,string>)
+addBrand(brand<long,string>)
+isDirectImporting() : boolean
+getAgreementNo() : long
+setAgreementNo(agreementNo : long)
+getTermsDate() : Date [2]
+setTermsDate(termsDate : Date [2])
+getDrivers() : Program_Drivers
+editProgram()
Item
-status : int
-item_Image: Object
-proforma : Proforma
-sales_repr : Sales_Repr
-customer : Member
-memberName
-brand : Object<long, string>
-division : Object<long, string>
+Item(desc:string, id:long, div:Object<long, string>)
+editItem()
+getBrand() : Object<long, string>
+setBrand(name:string, id:long)
+getDivision() : Object<long, string>
+setDivision(name:string, id:long)
+getSource() : Object<long, string>
+setSource(source : Object<long, string>)
+getStatus() : int
+setStatus(status : int)
+getSales_Repr() : Sales_Repr<long, string>
+setSale_Repr(repr:Sales_Repr)
+getProforma():Proforma
+deleteProforma(proforma:Proforma):boolean
+createProforma(item : Item, cust : Member)
+getLTBcustomer() : Member
+setLTBcustomer(val : Member)
+getItem_Image()
+setItem_Image(item_Image)
Inheritance
Inheritance
Inheritance
Inheritance
Member
Group
Member
Item


Multiple proformas can be generated for an item if the customer exceeds the standar limit, which is a year.
Program_Drivers
-royalty : double
-commission : double
-pickNpack : double
-preTicketing : double
-location : string
-landedCost : double
-programDrivers: Object<string,double>[]
-allDrivers : Set_Drivers
-customerCal : Customer
+Program_Drivers(v:Customer)
+getRoyalty() : double
+setRoyalty(royalty : double)
+getCommission() : double
+setCommission(val : double)
+getPickNpack() : double
+setPickNpack(val : double)
+getPreTicketing() : double
+setPreticketing(v : double)
+getLocation() : string
+setLocation(location : string)
+getLandedCost() : double
+setLandedCost(landedCost : double)
+getPrgDrivers() : Object<string,double>[]
+addDrivers(v:Object<string, double>[])
+editDrivers(v:Object<string, double>[])
+deleteDriver(v:Object<string, double>): boolean
+getCostOfGoods() : long
+TotalNetInvoice() : long
+getCostOfGoodsPlus() : long
+acrrualsNothers() : Accruals
+calculateMargin(drivers : Accruals) : long
Proforma
-items : Item[]
-proforma_id
-fillOrder : long
-retailPrice : long
-costOfGoods : long
-unitsPerDoor : long
-totalRetailPrice : long
-costOfGoodsPlus : long
-quantityCal : Volume
-customer : Member
-customerCal : Customer
–programDrivers : ProgramDrivers
+Proforma(v1 : Item, v2 : LTB_Customer)
+customerCal(quantityCal:Volume) : Customer
+quantityCal() : Volume
+programDrivers(v:Customer) : ProgramDrivers
Volume
-item : Item
-fillUnit : int
-doors : long
-anualQuantity : long
-quantityCal : Object<string, long>
+getFillUnit() : int
+setFillUnit(fillUnit : int)
+getDoors() : long
+setDoors(doors : long)
+setAnualQuantity(val : long)
+getAnualQuantity() : long
+getQuantity_Cal():Object<string,long>[]
Customer
-item : Item
-invoice : double
-margin : double
-volume : Volume
-customerCal : Object<string, double>[]
+Customer(vol : Volume)
+getInvoice() : double
+setInvoice(invoice : double)
+getMargin() : double
+setMargin(margin : double)
+getCustomer_Cal() : Object<string, double>[]
+total_Invoice() : long
+getUnitesPerDoors() : long
+totalRetail_Invoice() : long
Set_Drivers
-default_accruals : Driver[]
-default_nonAccruals: Driver[]
-Others_Accruals : Driver[]
-Others_NonAccruals : Driver[]
+Drivers()
-allDrivers = Driver[]
+addDefault_Accrual(driver : Driver)
+getDefault_Accrual(code : char[]) : Driver
+getDefault_Accruals() : Driver[]
+addDefault_NonAcrual(driver : Driver)
+getDefault_NonAccrual(code : char[]) : Driver
+getDefault_NonAccruals() : Driver[]
+addOther_Accrual(driver : Driver)
+getOther_Accrual(code : char[]) : Driver
+getOther_Accruals() : Driver[]
+addOther_NonAcrual(driver : Driver)
+getOther_NonAccrual(code : char[]) : Driver
+getOther_NonAccruals() : Driver[]
+getAllDrivers() : Driver[]
Driver
-SAP_Code : char[]
-name : string
-value : double
-lable : string
+Driver(name : string, value : double, code : char[])
+getName() : string
+setName(name : string)
+getValue() : double
+setValue(value : double)
+setSAP_Code(code : char[])
+getSAP_Code() : char[]
+getLabel() : string
+setLabel(label : string)
Super_User Inheritance
Program
Cost
Quantity Calculation
Customer Calculation
Driver
Member
Drivers are added by a member with "settings" permissions. Default accruals and non-acruals are set/modified by any member with "Finance" permissions and modified by any member with "Sales" permissions. Permissions are assigned by a Super_User parent, or a member with the permission to assign these permissions


ABSTRACT OPERATIONS
SUPER USER
PROGRAM
ITEM
PROFORMA
GROUP
PROGRAM DRIVERS
PERMISSIONS ?
VOLUME CALCULATIONS
MEMBER
Permissions ?
CUSTOMER CALCULATIONS
ADD/SET DRIVERS
End